

Overview

A mid-sized technology company is experiencing unexpectedly high employee turnover. The HR department wants to proactively identify employees who are at risk of leaving (i.e., attrition) so that targeted retention strategies can be applied. You are provided with a simulated dataset of 3,500 employee records and 19 input features. Your task is to build a Logistic Regression classification model to predict the binary target variable Attrition.

```
In [66]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [67]: df_train = pd.read_csv('../ST4035/Data/Assignment 1/ST 4035 Employee Training.csv')
df_test = pd.read_csv('../ST4035/Data/Assignment 1/ST 4035 Employee Test.csv', index_col='EmployeeID')
```

```
In [68]: df_train.info()

<class 'pandas.DataFrame'>
RangeIndex: 3500 entries, 0 to 3499
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   EmployeeID                            3500 non-null   str
1   Age                                    3500 non-null   int64
2   Gender                                3500 non-null   str
3   Department                            3500 non-null   str
4   JobRole                               3500 non-null   str
5   YearsAtCompany                        3500 non-null   int64
6   MonthlyIncome                        3500 non-null   int64
7   DistanceFromHome                     3500 non-null   int64
8   NumCompaniesWorked                   3500 non-null   int64
9   OverTime                             3500 non-null   str
10  TrainingTimesLastYear                 3500 non-null   int64
11  WorkLifeBalance                       3500 non-null   int64
12  JobSatisfaction                       3500 non-null   int64
13  EnvironmentSatisfaction                3500 non-null   int64
14  RelationshipSatisfaction               3500 non-null   int64
15  JobInvolvement                        3500 non-null   int64
16  PerformanceRating                     3500 non-null   int64
17  YearsSinceLastPromotion                3500 non-null   int64
18  YearsWithCurrManager                  3500 non-null   int64
19  BusinessTravel                        3500 non-null   str
20  Attrition                             3500 non-null   int64
dtypes: int64(15), str(6)
memory usage: 574.3 KB
```

```
In [69]: df_train['EmployeeID'].nunique()
```

```
Out[69]: 3500
```

No missing values or duplicates.
20 columns and 3500 rows.

```
In [70]: df_train = pd.read_csv('../ST4035/Data/Assignment 1/ST 4035 Employee Training.csv', index_col='EmployeeID')
```

```
In [71]: df_train.describe().transpose()
```

Out[71]:

	count	mean	std	min	25%	50%	75%	max
Age	3500.0	39.083429	12.343188	18.0	28.0	39.0	50.00	60.0
YearsAtCompany	3500.0	19.682571	11.813333	0.0	10.0	19.0	30.00	40.0
MonthlyIncome	3500.0	10552.667429	5552.778238	1021.0	5732.0	10494.5	15392.75	19984.0
DistanceFromHome	3500.0	15.174286	8.298512	1.0	8.0	15.0	22.00	29.0
NumCompaniesWorked	3500.0	4.527143	2.861099	0.0	2.0	4.0	7.00	9.0
TrainingTimesLastYear	3500.0	3.024286	1.991618	0.0	1.0	3.0	5.00	6.0
WorkLifeBalance	3500.0	2.512571	1.124241	1.0	1.0	3.0	4.00	4.0
JobSatisfaction	3500.0	2.509429	1.121472	1.0	2.0	2.5	4.00	4.0
EnvironmentSatisfaction	3500.0	2.519143	1.129221	1.0	2.0	2.0	4.00	4.0
RelationshipSatisfaction	3500.0	2.494286	1.133663	1.0	1.0	2.0	4.00	4.0
JobInvolvement	3500.0	2.496857	1.121252	1.0	1.0	2.0	4.00	4.0
PerformanceRating	3500.0	3.499714	0.500071	3.0	3.0	3.0	4.00	4.0
YearsSinceLastPromotion	3500.0	7.395714	4.598760	0.0	3.0	7.0	11.00	15.0
YearsWithCurrManager	3500.0	8.412571	5.205529	0.0	4.0	8.0	13.00	17.0
Attrition	3500.0	0.019714	0.139037	0.0	0.0	0.0	0.00	1.0

The Dataset consists of 3500 employees and average age of an employee is around 39 years with standard deviation around 12 years. While monthly income shows broad variability with mean of 10552.7 and standard deviation of 5552.8 indicating range of job roles and seniority levels. Also employees have stayed at this particular company for around 20 years on average indicating less attrition among employees.

In [72]:

```
# Train test split
from sklearn.model_selection import train_test_split

X = df_train.drop('Attrition', axis=1)
y = df_train['Attrition']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

EDA

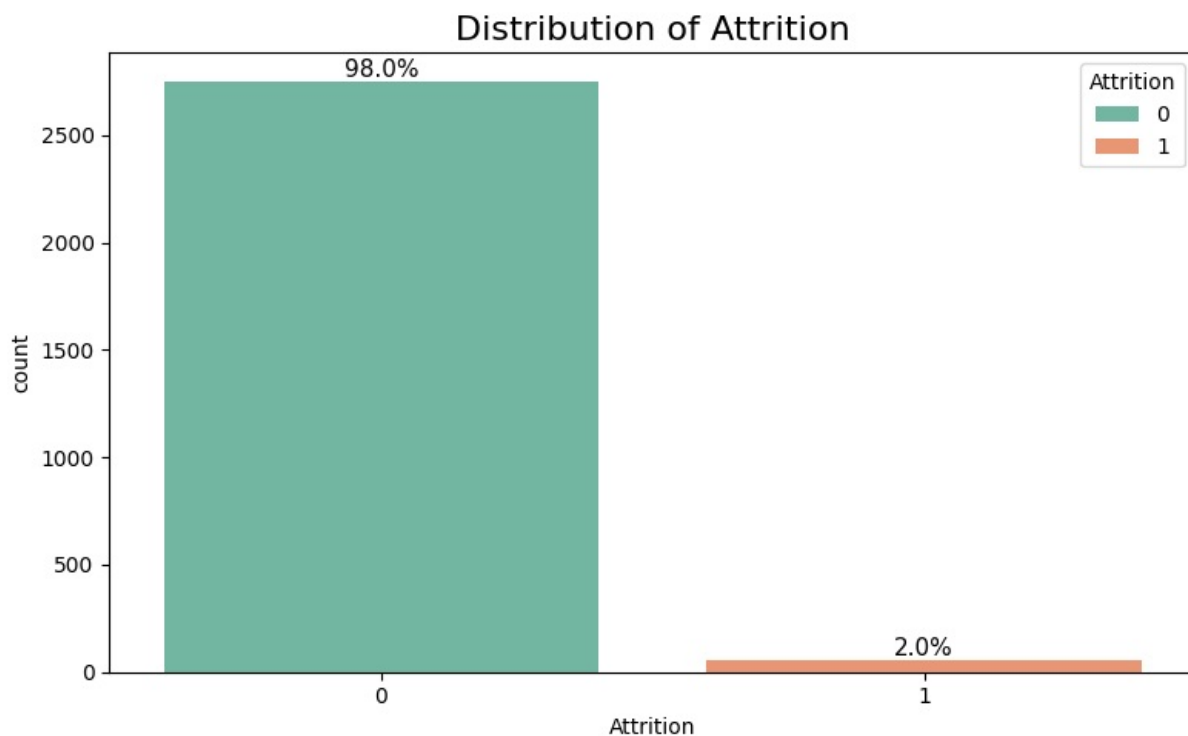
Univariate Analysis

In [73]:

```
plt.figure(figsize=(8, 5))
ax = sns.countplot(x="Attrition", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=False)

total = len(y_train)
for container in ax.containers:
    ax.bar_label(container, labels=[f'{v/total*100:.1f}%' for v in container.datavalues], fontsize=11)

plt.title('Distribution of Attrition', fontsize=16)
plt.tight_layout()
plt.show()
```



Target variable which is attrition is significantly imbalanced, around 98.0% of the target variable indicating no attrition while around 2% of the target variable indicating attrition.

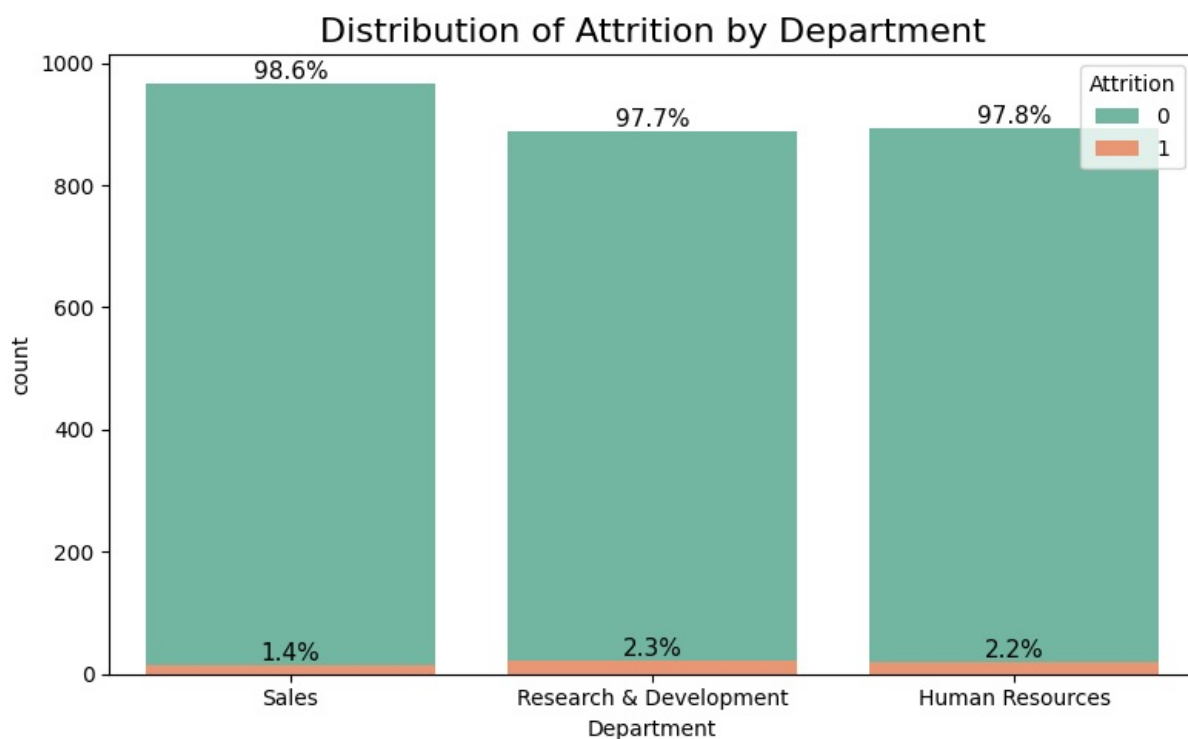
Bivariate Analysis

```
In [74]: plt.figure(figsize=(8, 5))
ax = sns.countplot(x="Department", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=False)

department_order = [tick.get_text() for tick in ax.get_xticklabels()]
total_per_department = X_train.join(y_train).groupby("Department")["Attrition"].count()

for container in ax.containers:
    labels = [f'{v/total_per_department[department]*100:.1f}%' for department, v in zip(department_order, container)]
    ax.bar_label(container, labels=labels, fontsize=11)

plt.title('Distribution of Attrition by Department', fontsize=16)
plt.tight_layout()
plt.show()
```



Research and development department shows the highest attrition rate and also it's the smallest department in size. Indicating employees from research and development department tend to leave the company more compared to other

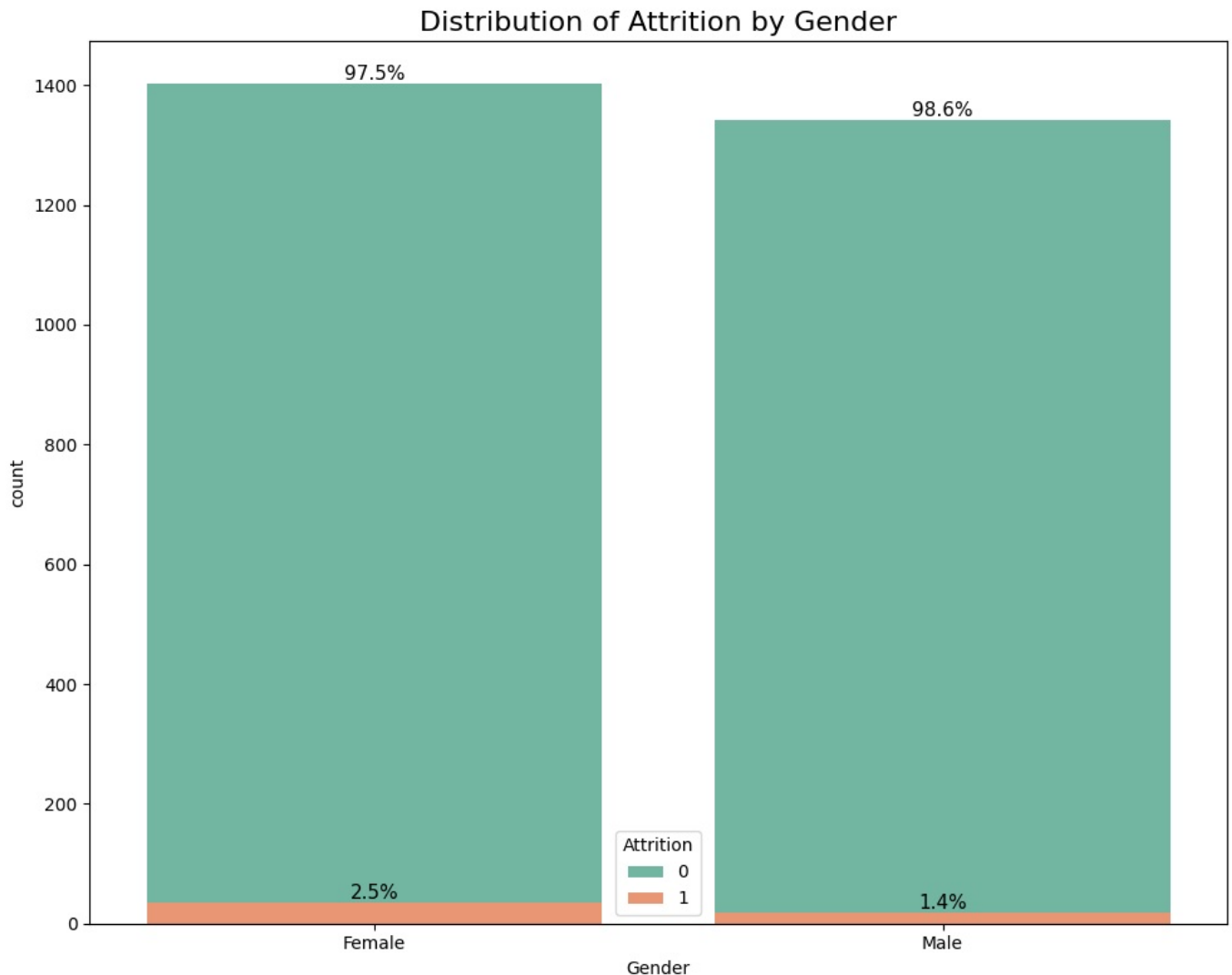
departments.

```
In [75]: plt.figure(figsize=(10, 8))
ax = sns.countplot(x="Gender", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=False)

gender_order = [tick.get_text() for tick in ax.get_xticklabels()]
total_per_gender = X_train.join(y_train).groupby("Gender")["Attrition"].count()

for container in ax.containers:
    labels = [f'{v/total_per_gender[gender]*100:.1f}%' for gender, v in zip(gender_order, container.datavalues)]
    ax.bar_label(container, labels=labels, fontsize=11)

plt.title('Distribution of Attrition by Gender', fontsize=16)
plt.tight_layout()
plt.show()
```



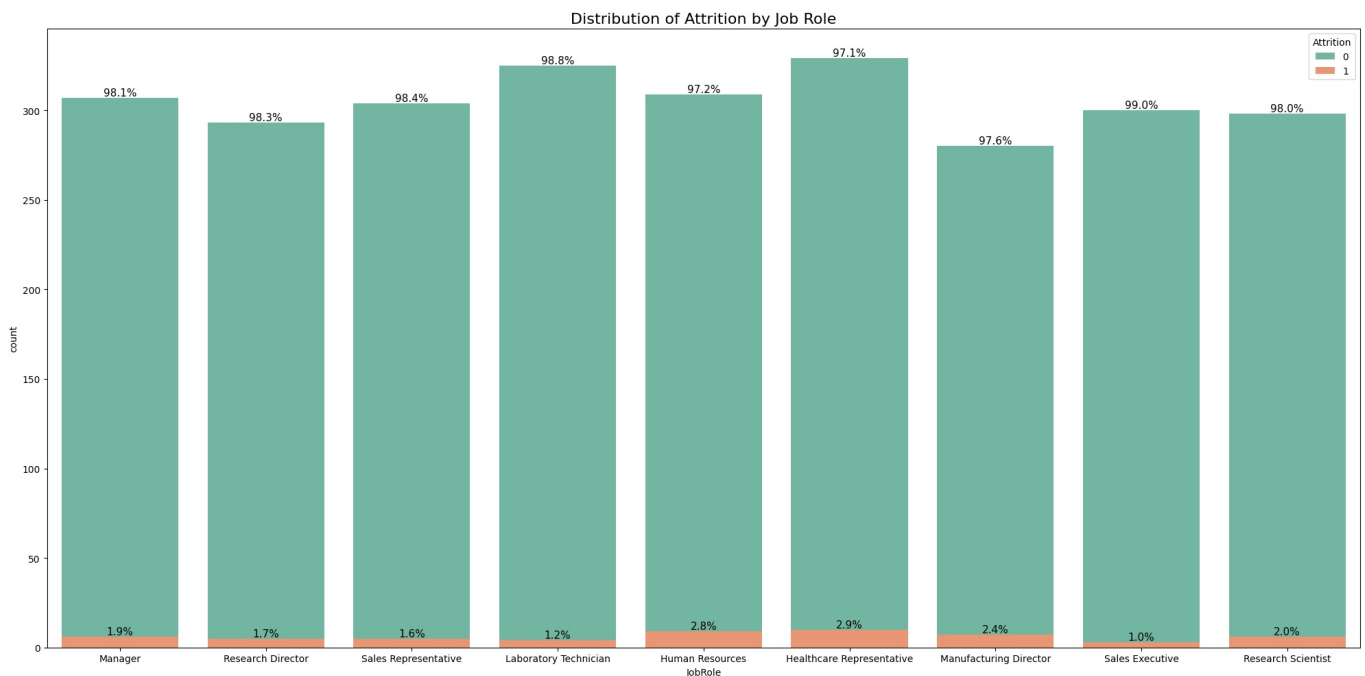
And when we visualize attrition rates by gender we can see that majority of the employees are females, and they are more likely to leave the company which is 2.5% of the total female employees.

```
In [76]: plt.figure(figsize=(20, 10))
ax = sns.countplot(x="JobRole", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=False)

jobrole_order = [tick.get_text() for tick in ax.get_xticklabels()]
total_per_jobrole = X_train.join(y_train).groupby("JobRole")["Attrition"].count()

for container in ax.containers:
    labels = [f'{v/total_per_jobrole[jobrole]*100:.1f}%' for jobrole, v in zip(jobrole_order, container.datavalues)]
    ax.bar_label(container, labels=labels, fontsize=11)

plt.title('Distribution of Attrition by Job Role', fontsize=16)
plt.tight_layout()
plt.show()
```



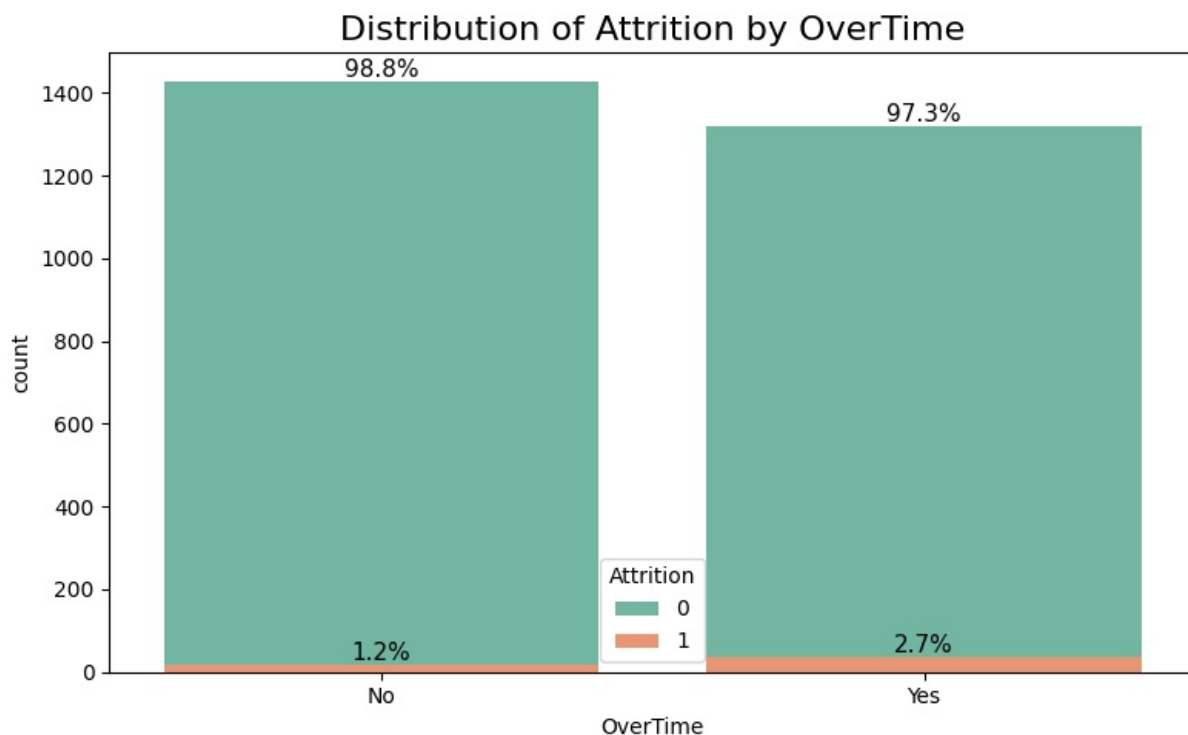
Healthcare Representatives has the highest number of employees in count and they have the highest percentage of attrition as well. While human resource employees are less than healthcare representatives in count they have the second highest attrition rate among all the job roles. which is around 2.8% .

```
In [77]: plt.figure(figsize=(8, 5))
ax = sns.countplot(x="OverTime", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=False)

overtime_order = [tick.get_text() for tick in ax.get_xticklabels()]
total_per_overtime = X_train.join(y_train).groupby("OverTime")["Attrition"].count()

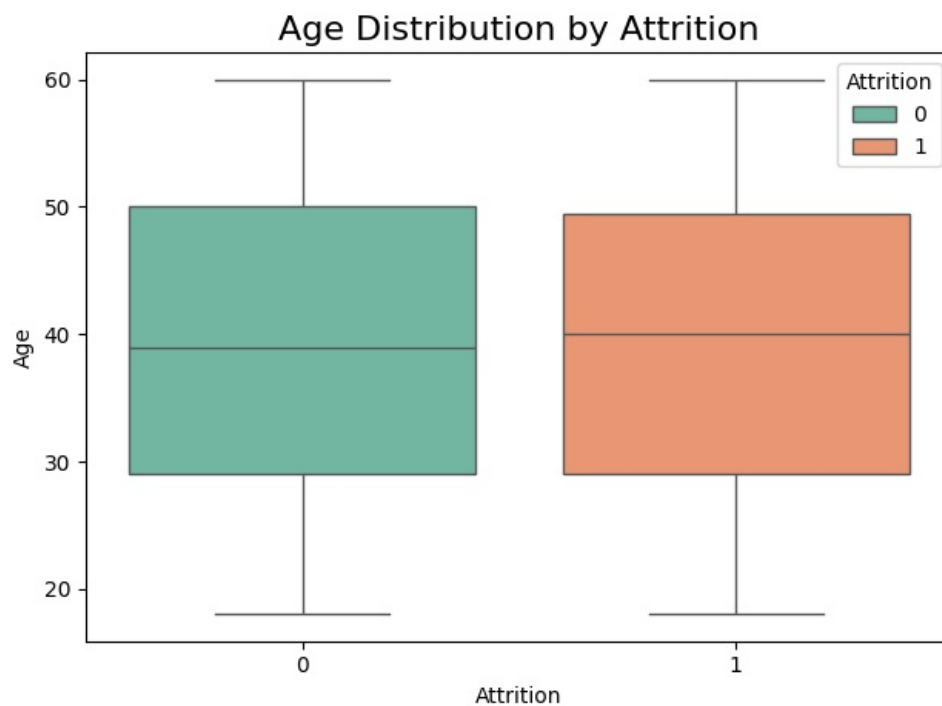
for container in ax.containers:
    labels = [f'{v/total_per_overtime[overtime]*100:.1f}%' for overtime, v in zip(overtime_order, container.data)]
    ax.bar_label(container, labels=labels, fontsize=11)

plt.title('Distribution of Attrition by OverTime', fontsize=16)
plt.tight_layout()
plt.show()
```



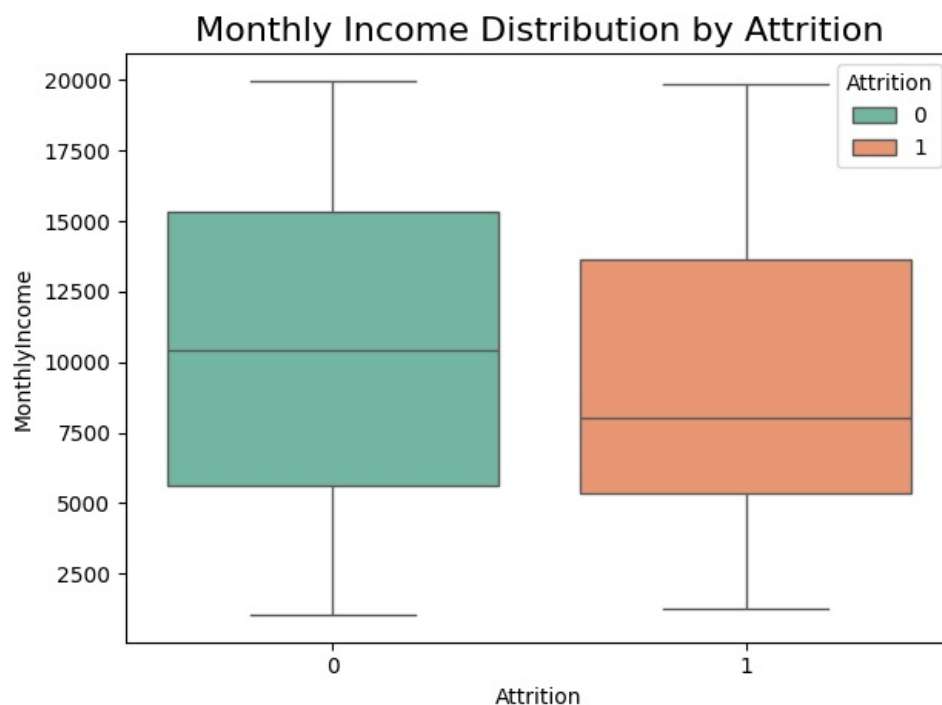
Among the people who are doing overtime 2.7% of them has left the company while 1.2% of the employees who do not do overtime has left the company.

```
In [78]: sns.boxplot(x="Attrition", y="Age", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=False)
plt.title('Age Distribution by Attrition', fontsize=16)
plt.tight_layout()
plt.show()
```



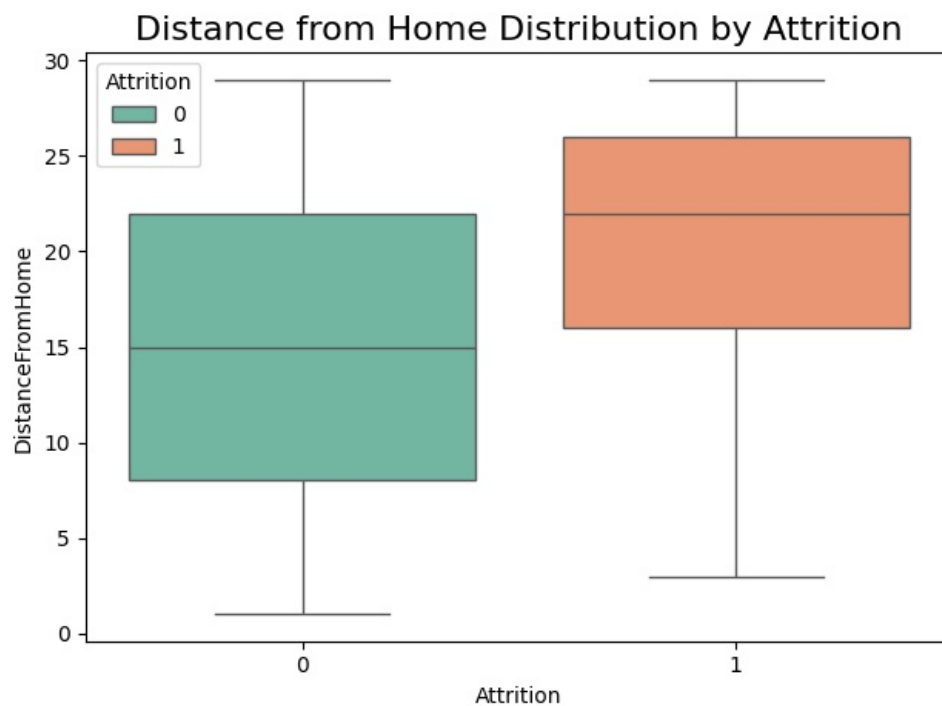
From this boxplot we can see that age haven't played significant role when employees leaving the company. And for both people who have left the company and who have stayed median age is around same and the age range is also similar.

```
In [79]: sns.boxplot(x="Attrition", y="MonthlyIncome", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=True)
plt.title('Monthly Income Distribution by Attrition', fontsize=16)
plt.tight_layout()
plt.show()
```



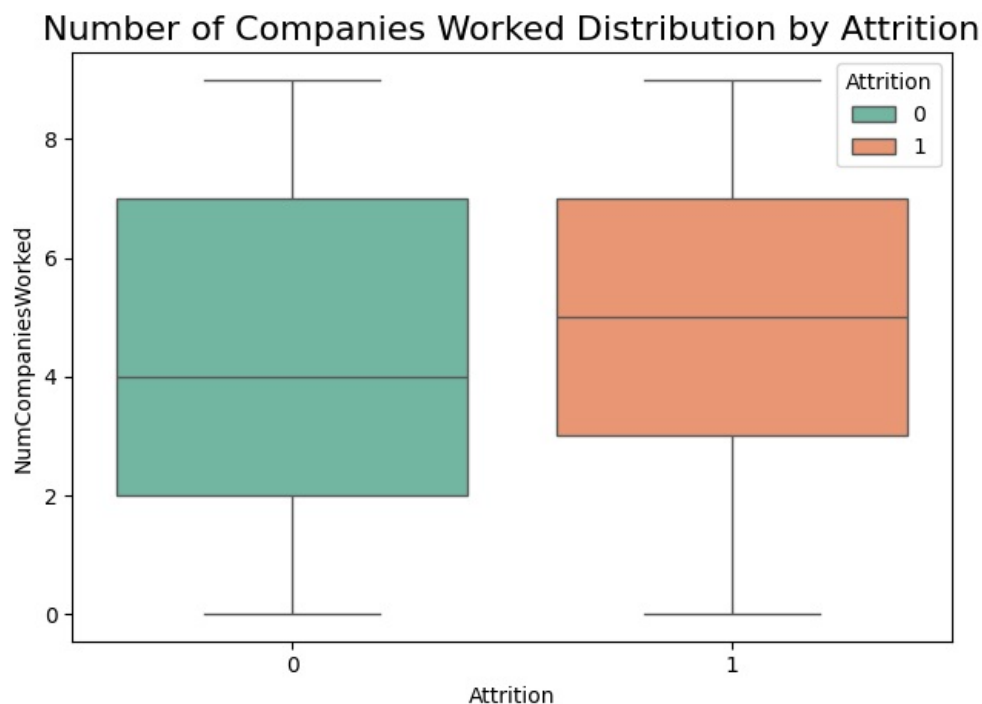
Not like age monthly income has affected whether an employee is going to stay or not, because among the employees who have left the company they have low median monthly income compared to people who have stayed.

```
In [80]: sns.boxplot(x="Attrition", y="DistanceFromHome", data=X_train.join(y_train), palette='Set2', hue='Attrition', dodge=True)
plt.title('Distance from Home Distribution by Attrition', fontsize=16)
plt.tight_layout()
plt.show()
```



Much like monthly income, distance from home has made noticeable impact on whether an employee will leave the company or not. As we can observe higher the distance from home you are more likely to leave the company.

```
In [81]: sns.boxplot(x="Attrition", y="NumCompaniesWorked", data=X_train.join(y_train), palette='Set2', hue='Attrition',
plt.title('Number of Companies Worked Distribution by Attrition', fontsize=16)
plt.tight_layout()
plt.show()
```

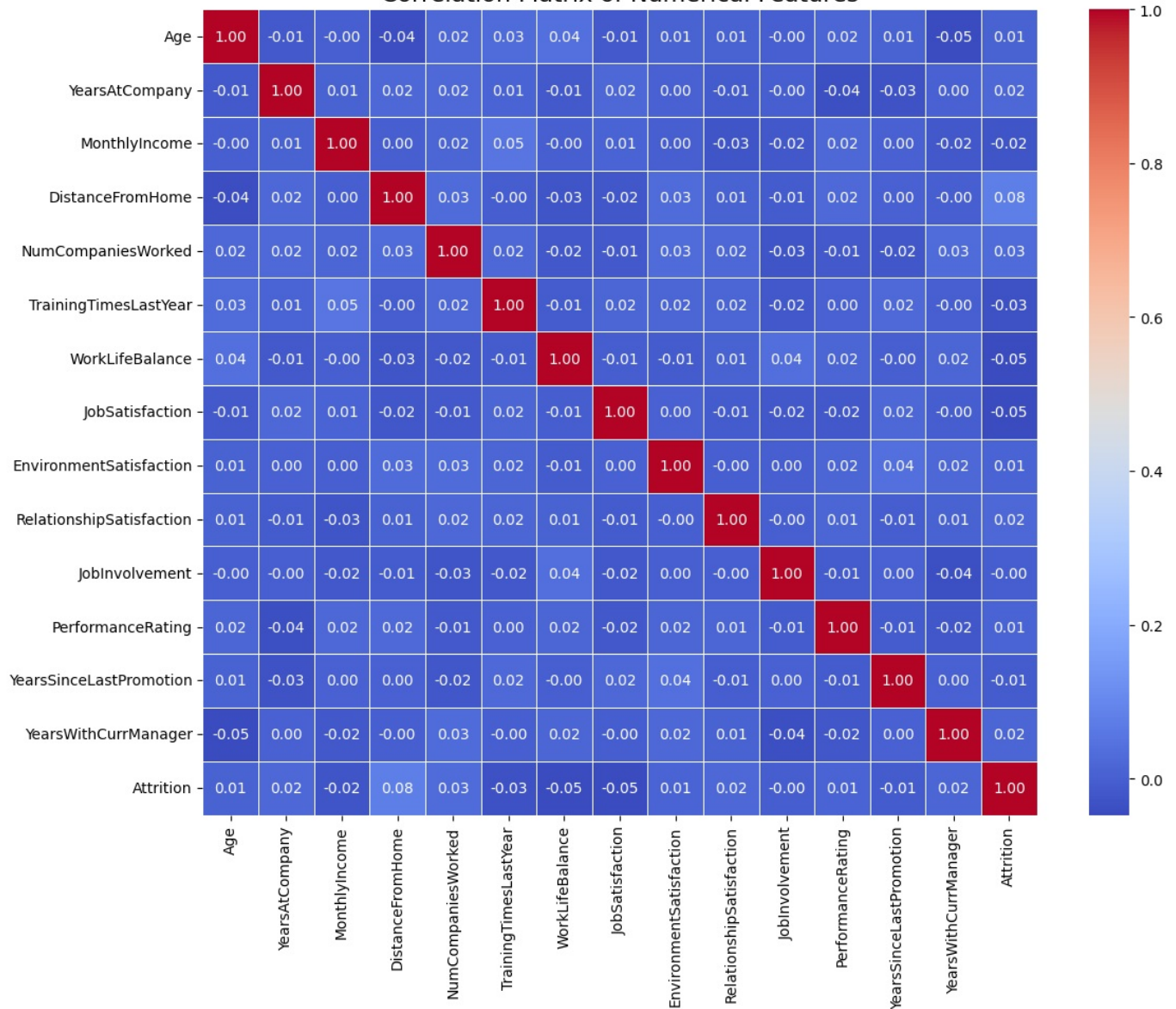


Number of companies worked seems to have affected the attrition, but 4 is the median for people who have stayed while 5 is the median for people who have left. While medians are not telling a convincing story we can see that if a person has higher number of companies he has worked for he has slightly more chance of leaving this company too.

Multivariate Analysis

```
In [82]: corr_matrix = X_train.join(y_train).select_dtypes(include=[np.number]).corr()
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', cbar=True, linewidths=0.5)
plt.title('Correlation Matrix of Numerical Features', fontsize=16)
plt.tight_layout()
plt.show()
```

Correlation Matrix of Numerical Features



```
In [83]: print("Top factors correlated with Attrition:")
print(corr_matrix['Attrition'].sort_values(ascending=False).head(6))
```

```
Top factors correlated with Attrition:
Attrition          1.000000
DistanceFromHome   0.076448
NumCompaniesWorked 0.029394
RelationshipSatisfaction 0.021592
YearsAtCompany     0.020152
YearsWithCurrManager 0.016682
Name: Attrition, dtype: float64
```

```
In [84]: print("Factors that make people STAY (Negative Correlation):")
print(corr_matrix['Attrition'].sort_values(ascending=True).head(5))
```

```
Factors that make people STAY (Negative Correlation):
WorkLifeBalance    -0.047308
JobSatisfaction     -0.045997
TrainingTimesLastYear -0.033719
MonthlyIncome      -0.023552
YearsSinceLastPromotion -0.007198
Name: Attrition, dtype: float64
```

From the correlation matrix we can see that there is no significant correlation among attrition and other variables. But among those variables distance from home has the highest correlation with attrition which is 0.076 and worklife balance has the highest negative correlation. which is -0.047.

```
In [85]: #Feature Engineering

def engineer_features(df):
    df = df.copy()

    df['TenurePerCompany'] = df['YearsAtCompany'] / (df['NumCompaniesWorked']+1)
    df['PromotionStagnation'] = df['YearsSinceLastPromotion'] / (df['YearsAtCompany']+1)
    df['ManagerStability'] = df['YearsWithCurrManager'] / (df['YearsAtCompany']+1)
    df = df.drop(columns=['YearsAtCompany', 'NumCompaniesWorked', 'YearsSinceLastPromotion', 'YearsWithCurrManager'])
```

```
return df
```

```
X_train = engineer_features(X_train)
X_val    = engineer_features(X_val)
df_test  = engineer_features(df_test)
```

```
In [86]: categorical_cols = X_train.select_dtypes(include=['object']).columns.tolist()
```

/var/folders/3f/l68d1j3x3598k23pcnmc2c440000gn/T/ipykernel_91465/823054385.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
categorical_cols = X_train.select_dtypes(include=['object']).columns.tolist()
```

```
In [87]: X_train_encoded = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True)
encoded_columns = X_train_encoded.columns
```

```
X_val_encoded = pd.get_dummies(X_val, columns=categorical_cols, drop_first=True).reindex(columns=encoded_columns)
df_test_encoded = pd.get_dummies(df_test, columns=categorical_cols, drop_first=True).reindex(columns=encoded_columns)
```

```
In [88]: test_index = df_test.index
```

```
In [89]: from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_val_scaled = scaler.transform(X_val_encoded)
X_test_scaled = scaler.transform(df_test_encoded)
```

```
In [90]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, f1_score
```

```
In [91]: from mlxtend.feature_selection import SequentialFeatureSelector as SFS
```

```
logit_fs = LogisticRegression(class_weight='balanced', random_state=42, max_iter=1000)
```

```
sfs = SFS(logit_fs,
          k_features='best',
          forward=True,
          floating=False,
          scoring='f1',
          cv=5,
          n_jobs=-1,
          verbose=1)
```

```
sfs.fit(X_train_scaled, y_train)
```

```
selected_idx = list(sfs.k_feature_idx_)
feature_names = X_train_encoded.columns.tolist()
selected_names = [feature_names[i] for i in selected_idx]
```

```
print(f"\nSelected {len(selected_idx)} out of {X_train_scaled.shape[1]} features:")
print(selected_names)
```

```
X_train_scaled = X_train_scaled[:, selected_idx]
X_val_scaled = X_val_scaled[:, selected_idx]
X_test_scaled = X_test_scaled[:, selected_idx]
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 27 out of 27 | elapsed: 2.0s finished
Features: 1/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 26 out of 26 | elapsed: 0.1s finished
Features: 2/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 25 out of 25 | elapsed: 0.1s finished
Features: 3/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 24 out of 24 | elapsed: 0.1s finished
Features: 4/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 23 out of 23 | elapsed: 0.1s finished
Features: 5/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 22 out of 22 | elapsed: 0.1s finished
Features: 6/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 21 out of 21 | elapsed: 0.1s finished
Features: 7/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 20 out of 20 | elapsed: 0.1s finished
Features: 8/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 19 out of 19 | elapsed: 0.1s finished
Features: 9/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 18 out of 18 | elapsed: 0.1s finished
Features: 10/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 17 out of 17 | elapsed: 0.1s finished
Features: 11/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 16 out of 16 | elapsed: 0.1s finished
Features: 12/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 15 out of 15 | elapsed: 0.1s finished
Features: 13/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 14 out of 14 | elapsed: 0.1s finished
Features: 14/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 13 out of 13 | elapsed: 0.1s finished
Features: 15/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 10 out of 12 | elapsed: 0.1s remaining: 0.0s
[Parallel(n_jobs=-1)]: Done 12 out of 12 | elapsed: 0.1s finished
Features: 16/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 8 out of 11 | elapsed: 0.1s remaining: 0.0s
[Parallel(n_jobs=-1)]: Done 11 out of 11 | elapsed: 0.1s finished
Features: 17/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 6 out of 10 | elapsed: 0.1s remaining: 0.1s
[Parallel(n_jobs=-1)]: Done 10 out of 10 | elapsed: 0.2s finished
Features: 18/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 4 out of 9 | elapsed: 0.1s remaining: 0.1s
[Parallel(n_jobs=-1)]: Done 9 out of 9 | elapsed: 0.1s finished
Features: 19/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 2 out of 8 | elapsed: 0.1s remaining: 0.2s
[Parallel(n_jobs=-1)]: Done 8 out of 8 | elapsed: 0.1s finished
Features: 20/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 7 out of 7 | elapsed: 0.1s finished
Features: 21/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 6 out of 6 | elapsed: 0.1s finished
Features: 22/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 2 out of 5 | elapsed: 0.0s remaining: 0.1s
[Parallel(n_jobs=-1)]: Done 5 out of 5 | elapsed: 0.1s finished
Features: 23/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 4 out of 4 | elapsed: 0.0s finished
Features: 24/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 3 out of 3 | elapsed: 0.0s finished
Features: 25/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 2 out of 2 | elapsed: 0.0s finished
Features: 26/27[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
Features: 27/27
```

Selected 9 out of 27 features:

```
['DistanceFromHome', 'WorkLifeBalance', 'JobSatisfaction', 'RelationshipSatisfaction', 'Department_Sales', 'JobRole_Manager', 'JobRole_Research Scientist', 'OverTime_Yes', 'BusinessTravel_Travel Frequently']
```

```
In [92]: logit = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
logit.fit(X_train_scaled, y_train)
```

```
Out[92]: ▼ LogisticRegression ⓘ ⓘ
          ► Parameters
```

```
In [93]: y_val_proba = logit.predict_proba(X_val_scaled)[:, 1]

thresholds = np.arange(0.1, 0.9, 0.01)
f1_scores = [f1_score(y_val, (y_val_proba >= t).astype(int)) for t in thresholds]

best_thresh = thresholds[np.argmax(f1_scores)]
print(f"Best threshold based on F1 score: {best_thresh:.2f}")
```

Best threshold based on F1 score: 0.70

```
In [94]: y_val_pred_tuned = (y_val_proba >= best_thresh).astype(int)
print("\nValidation Results")
print(classification_report(y_val, y_val_pred_tuned))
print(confusion_matrix(y_val, y_val_pred_tuned))

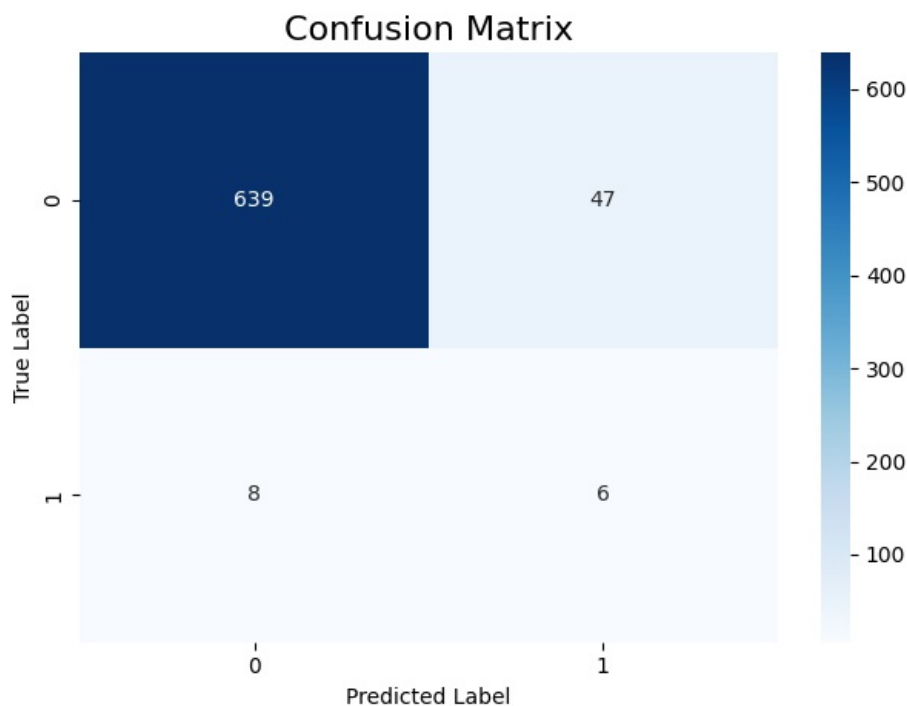
conf_matrix = confusion_matrix(y_val, y_val_pred_tuned)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix', fontsize=16)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()
```

```
Validation Results
              precision    recall  f1-score   support

     0       0.99         0.93         0.96         686
     1       0.11         0.43         0.18          14

 accuracy          0.92         0.92         0.92         700
 macro avg         0.55         0.68         0.57         700
 weighted avg         0.97         0.92         0.94         700
```

```
[[639  47]
 [  8   6]]
```



```
In [95]: coef_df = pd.DataFrame({
    'Feature': selected_names,
    'Coefficient': logit.coef_[0],
    'Odds Ratio': np.exp(logit.coef_[0])
}).sort_values('Coefficient', ascending=False)

print(coef_df.to_string(index=False))
```

	Feature	Coefficient	Odds Ratio
	DistanceFromHome	1.964081	7.128358
	OverTime_Yes	0.760282	2.138880
	RelationshipSatisfaction	0.416168	1.516141
BusinessTravel_Travel_Frequently	0.174189	1.190280	
JobRole_Manager	-0.058551	0.943130	
JobRole_Research Scientist	-0.200485	0.818334	
Department_Sales	-0.464484	0.628459	
JobSatisfaction	-0.987785	0.372401	
WorkLifeBalance	-0.996576	0.369141	

log-

odds(Attrition) = $\beta_0 + 1.964 \cdot \text{DistanceFromHome} + 0.760 \cdot \text{OverTime_Yes} + 0.416 \cdot \text{RelationshipSatisfaction} + 0.174 \cdot \text{Busir}$
 $- 0.059 \cdot \text{JobRole_Manager} - 0.200 \cdot \text{JobRole_ResearchScientist} - 0.464 \cdot \text{Department_Sales} - 0.988 \cdot \text{JobSatisfaction} - 0.9$

When DistanceFromHome increase by 1 unit, log ods of attrition increase by 1.96

Odds of attrition is 2.1 times moref or employees who are doing overtime compared to odds of attrition for employees who do not do overtime

when BusinessTravel_Travel_Frequently increase by 1 unit, log ods of attrition increase by 0.17

Odds of attrition is 1.1 times less for employees with JobRole Manager, compared to odds of attrition for employees with other job roles

Odds of attrition is 1.2 times less for employees with JobRole Research Scientist, compared to odds of attrition for employees with other job roles

Odds of attrition is 1.6 times less for employees in JobRole Department Sales, compared to odds of attrition for employees with other Departments

when JobSatisfaction increase by 1 unit, log odds of attrition decrease by 0.99

when WorkLifeBalance increase by 1 unit, log odds of attrition decrease by 1

```
In [96]: y_test_proba = logit.predict_proba(X_test_scaled)[: , 1]
y_test_pred = (y_test_proba >= best_thresh).astype(int)

submission = pd.DataFrame({
    'EmployeeID': test_index,
    'Attrition' : y_test_pred
})

submission.to_csv('submission.csv', index=False)
```